

APIs

Profesor: Pablo J Piccoli Bornia

Auxiliares: Daniela Longas - M Victoria Tolosa

COMUNICACIÓN REMOTA

Comunicación remota



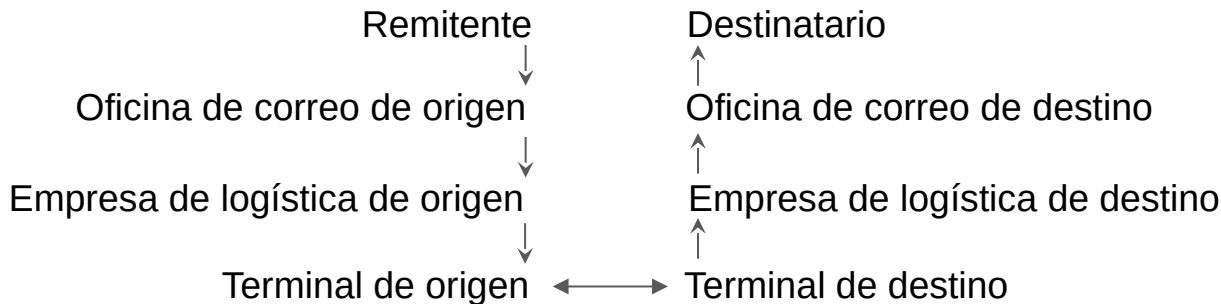
En la actualidad, la mayoría de los programas que usamos se comunican con otros programas que corren en otros dispositivos:

chat, páginas web, audio, e-commerce, geolocalización, redes sociales, juegos, videoconferencias, etc.

¿Cómo sabe una app cuál es el clima actual? ¿Cómo obtiene Instagram los posts que ves?

Comunicación remota

Ejemplo: Correo Postal



- En el origen, cada nivel delega la responsabilidad del envío en el nivel inferior.
- En el destino, cada nivel le avisa al nivel superior que “le llegó algo qué tiene que entregar”

Cada nivel maneja los mismos **protocolos** (hablan el mismo idioma) que su contraparte remota

Modelo TCP/IP

El modelo **TCP/IP** (Transmission Control Protocol/Internet Protocol) es un modelo de capas que define un conjunto de protocolos para **transmitir** datos entre distintas computadoras a través de una red

Remitente → servidor

Carta → request

Respuesta → response

Capas → modelo TCP/IP

Capa de aplicación	↔	Capa de aplicación
Capa de transporte	↔	Capa de transporte
Capa de red	↔	Capa de red
Capa de interfaz de red	↔	Capa de interfaz de red

Protocolos:

Aplicación: HTTP, SMTP, Telnet, FTP, etc.

Transporte: TCP, UDP, etc.

Red: IP, ARP, IGMP, ICMP, etc.

Interfaz de red: ETHERNET, L2TP, etc.

PROTOCOLLO HTTP

Protocolo HTTP

HTTP (*Hypertext Transfer Protocol*) es un protocolo de comunicación que permite las transferencias de información a través de archivos en Internet

**¿Qué es lo que escribimos en
la barra de direcciones de un browser?**

URL (*Uniform Resource Locator*) es una referencia a un recurso que especifica su ubicación en una red informática y un protocolo para recuperarlo



protocolo://host/carpeta1/carpeta2/.../carpetan/recurso

Protocolo HTTP

protocolo://host/carpeta₁/carpeta₂/.../carpeta_N/recurso

Ejemplo:

<https://www.ole.com.ar/tema/lionel-messi.html>

Protocolo: https

Host: www.ole.com.ar

Ubicación en el host: tema/

Recurso: archivo lionel-messi.html

Nota: *El protocolo https se diferencia de http en que encripta los datos antes de enviarlos por la red.*

Protocolo HTTP

¿Qué información podemos pedir usando HTTP?

Archivos (estático)

- HTML
- Imagen
- TXT
- PDF

Información dinámica

- Clima actual
- Últimos posts
- Productos
- Saldo bancario

- Un archivo ya existe en el servidor.
- Si yo pido el clima actual el servidor tiene que generar esa información en el momento

BIBLIOTECA REQUESTS

Biblioteca Requests

→ *Para comunicarnos con hosts remotos y pedirles recursos*

```
import requests
def obtener_texto () :
    url = " https :// www .... "
    response = requests . get ( url )
    return response . text
input (" Presiona Enter para descargar el texto ")
texto = obtener_texto ()
print ("\ nTexto recibido :")
print ( texto )
```

- La función `requests.get(url)` envía una solicitud HTTP para obtener el recurso indicado por url y devuelve la respuesta del servidor.
- El resultado es un objeto, instancia de la clase requests. ***Response***.
- Los objetos *Response* tienen un atributo `text` que contiene el texto devuelto por el servidor.

Biblioteca Requests

→ *En muchos casos, la respuesta del servidor remoto no está pensada para ser leída directamente por una persona.*

Ejemplo: los archivos con extensión .html (o .htm) son textos que los navegadores web (como Chrome o Firefox) interpretan para mostrar una página en pantalla.

```
import requests

x = requests . get ('https :// w3schools . com / python / demopage . htm
')
print ( x.text )
```

Biblioteca Requests

→ En muchos casos, no se solicita un archivo específico, sino que se invoca una funcionalidad provista por una **API** (Application Programming Interface) que corre en el servidor remoto.

API → biblioteca de funciones disponibles para su uso cuya respuesta depende de cómo haya sido programada, sin seguir una estructura fija.

```
import requests
def obtener_chiste () :
    url = " https:// official - joke - api . appspot . com / random_joke "
    response = requests . get ( url )
    return response . text
```

```
input (" Presiona Enter para obtener un chiste")
chiste = obtener_chiste ()
print ("\ nChiste recibido :")
print ( chiste)
```

```
Chiste recibido : {"
type ":" general ", "
setup ":" Want to hear
a joke about
construction ?",
" punchline ":" Nah ,
I'm still working on
it .",
" id ":151}
```

Biblioteca Requests

*Es muy habitual que el resultado de una función de una API tenga formato **JSON**.*

*La clase Response tiene un método **res.json()** → en caso de que la respuesta de la API tenga este formato, transforma su contenido en objetos Python del mismo modo que lo hacía la biblioteca **json**.*

```
import requests
def obtener_chiste () :
    url = " https :// official - joke - api . appspot . com /
random_joke "
    response = requests . get ( url )
    data = response . json ()
    chiste = data [ " setup " ] + " " + data [ " punchline " ]
    return chiste
```

```
input ("Presiona Enter para obtener un chiste")
chiste = obtener_chiste ()
print ("\ nChiste recibido :")
print ( chiste)
```

Chiste recibido :
Why don 't you find
hippopotamuses hiding in
trees ? They 're really good at
it .

HTTP: parámetros

Cuando hacemos un requerimiento HTTP se pueden incluir parámetros y parámetros.

*→ al final de la URL se incluye el signo **?** seguido de pares **clave=valor** separados por el símbolo **&**.*

***Ejemplo:** accediendo a **https://pizzaloca.com/pedido?nro pedido=2** solicitamos el segundo pedido almacenado en el servidor:*

```
{
  "tamano": "grande",
  "precio": 4300.00,
  "adicionales": ["jamón", "morrones"],
  "cliente": {
    "nombre": "Nombre",
    "telefono": "1132234532",
    "correo": "nombre01@em.com"
  }
}
```

HTTP: parámetros

Cuando son muchos los parámetros que se envían en un HTTP, incluirlos todos en la URL resulta desprolijo.

Ejemplo: `https://pizzaloca.com/pedido?nro_pedido=2&fecha=12-03-98&cortada=True&aceitunas=False`

*La función **get(...)** admite un parámetro opcional **params**, para indicar en un diccionario de Python los valores de los parámetros que son enviados al servidor.
→ las claves como los valores deben ser **strings***

```
valores_params = dict ()
valores_params ['nro_pedido '] = '2'
valores_params ['fecha '] = '12 -03 -98 '
valores_params ['cortada '] = 'True '
valores_params ['aceitunas '] = 'False '
resp = requests . get ('https :// pizzaloca . com / pedido ', params = valores_params )
```

Repaso

- Redes
- Modelo de capas
- HTTP
- Biblioteca requests
- Consultas HTTP con parámetros